Internal FFT Utilities Specification

Shared Planning and Positive-Lag Correlation Primitives for mdstats

mdstats

2026-07-11

1. Purpose and status

This document specifies the internal module

`mdstats/analysis/_fft.py`

The module is not part of the public `mdstats` API. It centralizes numerical machinery shared by `msd.py`, `vacf.py`, and future time-correlation modules:

- linear-correlation zero padding;
- positive-lag pair counts;
- memory-aware atom-block planning;
- inversion of real cross spectra into positive-lag correlation sums.

Estimator-specific physics remains outside `_fft.py`. The module does not know whether a time series represents positions, velocities, currents, stresses, or another observable.

2. Design motives

The direct and FFT implementations of an observable must represent the same mathematical estimator. Duplicating padding, lag orientation, pair-count, and memory-planning logic in every analysis module would make subtle convention errors likely.

The shared layer therefore has a narrow boundary:

```
analysis-specific arrays and formulas
              |
              v
shared FFT planning and correlation inversion
              |
              v
analysis-specific normalization and result objects
```

This separation allows later reuse for current autocorrelations, stress correlations, intermediate scattering functions, and other uniformly sampled time-series observables without embedding VACF- or MSD-specific assumptions.

3. Mathematical convention

Let two real, uniformly sampled series be

$$x_n, \qquad y_n, \qquad n = 0, \ldots, T-1.$$

The positive-lag linear cross-correlation sum is defined as

$$R_{xy}(k) = \sum_{n=0}^{T-k-1} x_n y_{n+k}, \qquad k = 0, \ldots, T-1.$$

The corresponding cross spectrum uses

$$\hat{R}_{xy} = \mathrm{FFT}(x)^* \ \mathrm{FFT}(y).$$

After zero padding and inverse transformation,

$$R_{xy}(k) = \mathrm{IFFT}\left[\mathrm{FFT}\ (x)^*\ \mathrm{FFT}\ (y)\right]_k.$$

The order of the conjugation is part of the API contract. Reversing it changes which series is evaluated at the later time and transposes positive-lag tensor conventions.

The shared routine returns **unnormalized sums**. An estimator divides by its own pair counts, atom counts, masses, volumes, or other physical factors.

4. Linear rather than circular correlation

An FFT without sufficient padding produces a circular correlation. To recover the linear sum, the transform length must satisfy

$$N_{\mathrm{FFT}} \geq 2T - 1.$$

The implementation chooses

$$N_{\mathrm{FFT}} = \mathrm{next\_fast\_len}(2T - 1),$$

where `scipy.fft.next_fast_len()` selects a computationally favorable length. The real transform contains

$$N_f = \left\lfloor \frac{N_{\mathrm{FFT}}}{2} \right\rfloor + 1$$

frequency bins.

5. Internal data structure

```
@dataclass(frozen=True, slots=True)
class AtomFFTPlan:
    n_fft: int
    n_frequency: int
    atom_block_size: int
```

The plan records only numerical execution parameters. It does not contain physical selections, weights, lags, units, or normalization conventions.

6. Internal function contracts

6.1 `linear_fft_length()`

```
def linear_fft_length(n_samples: int) -> int:
    ...
```

Returns a fast transform length sufficient for linear correlation.

Constraints:

- `n_samples` must be an integer;
- `n_samples >= 1`;
- the returned value is at least `2 * n_samples - 1`.

6.2 `positive_lag_pair_counts()`

```
def positive_lag_pair_counts(
    n_samples: int,
    max_lag: int,
) -> NDArray[np.float64]:
    ...
```

For all-origin sampling, the number of valid pairs is

$$N_{\mathrm{pair}}(k) = T - k.$$

The returned array is

```
[T, T - 1, ..., T - max_lag]
```

as `float64`, because it is normally used directly in division.

Constraints:

- `n_samples >= 1`;
- `max_lag` is an integer;
- `0 <= max_lag < n_samples`.

This helper applies only to all-origin correlations. Sparse time-origin sampling must calculate its own pair counts in the direct estimator.

6.3 `make_atom_fft_plan()`

```
def make_atom_fft_plan(
    n_atoms: int,
    n_frames: int,
    *,
    atom_block_size: int | None = None,
    memory_target_bytes: int = 256 * 1024 * 1024,
    real_series_per_atom: int = 3,
    complex_series_per_atom: int = 3,
    inverse_real_series_per_atom: int = 3,
) -> AtomFFTPlan:
    ...
```

The caller describes the largest expected temporary workspace per atom. The planner estimates

$$B_{\mathrm{atom}} = 8N_rT + 16N_cN_f + 8N_iN_{\mathrm{FFT}},$$

where N_r, N_c, and N_i are the requested counts of real input, complex frequency, and inverse-real work arrays per atom.

If `atom_block_size` is omitted, the resolved block size is approximately

$$B = \min\left(N, \max\left[1, \left\lfloor \frac{M_{\mathrm{target}}}{B_{\mathrm{atom}}} \right\rfloor\right]\right).$$

The estimate is deliberately conservative; it is a planning heuristic rather than an exact peak-memory measurement.

Constraints:

- `n_atoms >= 1` and `n_frames >= 1`;
- the memory target is a positive integer;
- series counts are nonnegative integers;
- an explicit block size is a positive integer and is clipped to `n_atoms`.

6.4 `positive_lag_correlation_from_spectrum()`

```
def positive_lag_correlation_from_spectrum(
    cross_spectrum: NDArray[np.complex128],
    *,
    n_fft: int,
    max_lag: int,
) -> NDArray[np.float64]:
    ...
```

The last axis of `cross_spectrum` must contain a real-FFT spectrum of length

$$N_f = N_{\mathrm{FFT}}/2 + 1$$

using integer division. The routine applies `scipy.fft.irfft()` and returns lags `0:max_lag + 1` along the last axis.

The input must follow

```
np.conj(rfft(x, n=n_fft)) * rfft(y, n=n_fft)
```

so that the returned value at lag k is $\sum_n x_n y_{n+k}$.

The function does not divide by pair counts and does not clip roundoff.

7. High-level usage

```
validate a uniformly sampled trajectory
          |
construct selected real time series
          |
make_atom_fft_plan(...)
          |
for each atom block:
    rfft(real series, n=plan.n_fft)
    build estimator-specific auto/cross spectra
    accumulate spectra or retain per-atom spectra
          |
positive_lag_correlation_from_spectrum(...)
          |
divide by estimator-specific pair counts
          |
construct MSDResult, VACFResult, or another result type
```

VACF uses the primitive directly on velocity auto- and cross spectra. MSD uses it on position spectra and combines the resulting correlations with early- and late-time coordinate-product sums.

8. Deliberate non-goals

`_fft.py` does not:

- validate trajectory semantics or time grids;
- select atoms or species;
- subtract drift or temporal means;
- center coordinates for numerical stability;
- apply physical weights;
- choose `max_lag`, `lag_stride`, or sparse time origins;
- normalize by pair counts;
- window, smooth, integrate, or Fourier-transform a reported correlation;
- construct public result objects;
- choose between direct and FFT estimator backends.

Those decisions remain in the calling analysis module.

9. Numerical and edge-case warnings

9.1 Correlation orientation

The spectrum order must remain `conj(FFT(x)) * FFT(y)`. A swapped order may leave scalar autocorrelations unchanged while silently transposing off-diagonal time-lag tensors.

9.2 Zero padding

Using fewer than `2 * n_samples - 1` points causes wraparound contamination. The helper owns this invariant so callers should not substitute an arbitrary FFT length.

9.3 Pair-count normalization

`positive_lag_correlation_from_spectrum()` returns sums. Forgetting division by $T - k$ changes the estimator and attenuates or amplifies the tail depending on the chosen normalization.

9.4 Sparse origins

A conventional FFT autocorrelation uses every possible time origin. It cannot reproduce `origin_stride > 1` without a different masked-correlation formulation. Current modules therefore dispatch sparse-origin calculations to the direct backend.

9.5 Memory planning

The planner estimates major NumPy/SciPy arrays but cannot account exactly for allocator overhead, FFT work buffers, threading, or copies made by caller expressions. Callers may lower `atom_block_size` when operating near a memory limit.

9.6 Floating-point cancellation

The shared layer does not repair estimator-specific cancellation. For example, FFT MSD centers each atom's coordinate series before applying an expanded square identity. Negative roundoff handling remains in `msd.py`.

10. Required tests

The internal test contract should verify:

1. `linear_fft_length(T) >= 2T - 1` for odd and even T.
2. Pair counts equal `T - k` for every valid lag.
3. FFT autocorrelation matches direct sums for random real series.
4. Cross-correlation orientation matches $\sum_n x_n y_{n+k}$.
5. Leading dimensions are preserved for batched spectra.
6. Explicit and automatic atom block sizes are valid and bounded.
7. Invalid sample counts, lag ranges, memory targets, and spectrum shapes raise descriptive exceptions.
8. MSD and VACF direct/FFT equivalence tests remain the integration-level guard against convention drift.

11. Future expansion

The module may later support:

- shared backend cost models;
- configurable memory targets exposed through package settings;
- complex-valued time-series correlations;
- masked or segmented FFT correlations;
- Welch/block spectral helpers;
- reusable current, stress, dipole, or scattering-function correlations.

Any expansion should preserve the current positive-lag orientation and keep physical normalization outside the shared numerical layer.